

System Programming Project 4:

Allocator

담당 교수 : 김영재 교수님
이름 : 이석희
학번 : 20200595

1. 프로젝트 개요

본 프로젝트는 C 프로그램을 위한 동적 메모리 할당기(Dynamic Memory Allocator)를 직접 구현하는 과제로, 기존의 libc 라이브러리에 포함된 malloc, free, realloc 함수를 대체하는 mm_malloc, mm_free, mm_realloc 함수를 정의하고 최적화된 방식으로 구현하는 것을 목표로 한다. 이 과제를 통해 저수준 메모리 관리의 원리와 다양한 성능 최적화 기법을 학습하고, 메모리 단편화(fragmentation), 블록 병합(coalescing), 분할(splitting) 등의 문제를 해결하는 전략을 설계 및 구현한다.

본 구현은 **Segregated Free List** 기반의 메모리 할당기를 채택하였으며, 20 개의 크기 범주로 나누어진 리스트를 통해 빠르고 효율적인 블록 할당 및 해제를 지원한다. 또한 First-fit 전략과 블록 크기 정렬을 조합하여 탐색 성능을 개선하였다.

프로젝트의 목적

- C 언어로 메모리 할당기 구현을 통해 시스템 수준의 메모리 관리 이해
- 다양한 메모리 할당 전략 비교 및 최적화 기법 실습
- mdriver 를 통한 할당기의 정확성, 메모리 사용률, 처리량 평가

구현 대상 함수

- int mm_init(void)
- void *mm_malloc(size_t size)
- void mm_free(void *ptr)
- void *mm_realloc(void *ptr, size_t size)
- 기타 user defined functions

2. 전체 설계 구조

본 할당기는 **Segregated Free List** 를 기반으로 하여 메모리 블록을 크기 구간별로 분리된 여러 개의 연결 리스트에 관리함으로써 malloc, free, realloc 요청 시 효율적인 탐색과 병합(Coalescing)을 가능하게 설계되었다. 각 기능은 블록 관리의 정합성과 속도 향상을 위해 역할별로 분리된 서브루틴들에 의해 수행된다.

2.1 전체 구조 요약

- **초기화 (mm_init):**
메모리 공간의 초기 힙과 segregated free list 배열을 초기화하고, metadata(prologue, epilogue) block 을 설정하여 heap 의 경계를 관리한다.
- **할당 (mm_malloc):**
주어진 요청 크기를 ALIGN 후, 해당 크기에 적합한 free list 를 순차 탐색하여 적합한 block 을 찾아 할당한다. 적당한 크기가 없을 경우 heap 을 확장한 뒤 새 블록을 할당한다.
- **해제 (mm_free):**

- 사용자가 지정한 블록을 free 상태로 마킹한 뒤, 앞뒤 블록이 free 일 경우 병합을 수행하고, 병합 결과를 다시 free list 에 삽입한다.
- **재할당 (mm_realloc):**
기존 블록을 새 크기로 확장 가능한지 검사하고 가능하면 in-place 확장, 불가능할 경우 새로운 블록을 할당한 뒤 기존 내용을 복사하고 해제한다.

2.2 동작 흐름도 (Flowchart 의 크기가 커서 다음 페이지에 따로 넣었습니다.)

- **malloc() flow**
 1. 요청 크기를 ALIGN
 2. 적절한 segregated list 탐색
 3. 블록이 존재하면 분할 및 할당
 4. 없으면 heap 확장 후 다시 시도
- **free() flow**
 1. 블록을 free 로 설정
 2. 앞뒤 블록과 병합(coalescing)
 3. free list 에 삽입
- **realloc() flow**
 1. In-place: 새 크기가 기존보다 작거나 같으면 그대로 반환
 2. Expand: 뒤 블록이 free 이고 공간이 충분하면 확장
 3. 위 경우에 해당되지 않으면 새 블록을 할당하고 복사 후 기존 블록 해제

2.3 블록 구조

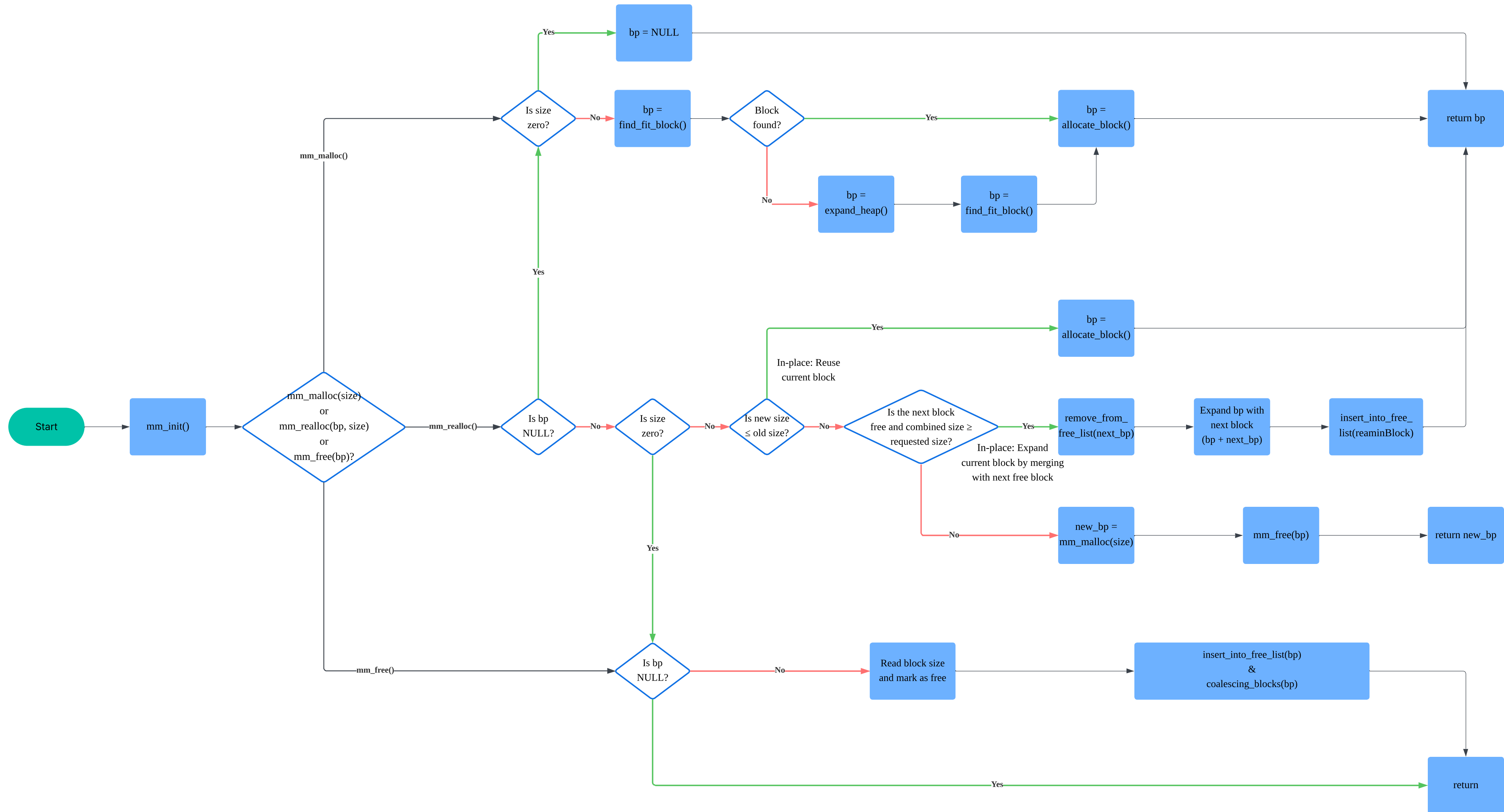
- HEADER: 4 bytes (size + alloc bit)
- FOOTER: 4 bytes (size + alloc bit)
- Free 블록: payload 내부에 prev, next 포인터 (double linked list)
- 최소 블록 크기: 32 bytes
 - = ALIGN(HEADER(4B) + padding(4B) + prev(8B) + next(8B) + FOOTER(4B))

2.4 Segregated List 구성

- 총 20 개의 list
- 각각의 list 는 블록 크기 범위별로 다음과 같이 설정됨

List Index	Block Size 범위
Index 0	1 ~ 32 bytes
Index 1	33 ~ 64 bytes
Index 2	65 ~ 128 bytes
...	...
Index 19	2 ¹⁹ bytes (최대 크기)

각 list 는 오름차순으로 정렬되며 First-Fit 탐색 전략을 사용하여 가장 앞에서부터 적절한 블록을 선택함.



3. mm.c 및 주요 구현 함수 설명

본 섹션에서는 mm.c 파일에 정의된 라이브러리, 매크로, 전역변수, subroutine 들의 목적, 동작 방식 및 상호작용을 서술한다.

3.1 헤더 파일

- `#include <stdio.h>`: 표준 입출력 기능을 제공하는 헤더
- `#include <stdlib.h>`: 표준 라이브러리 함수 및 자료형을 사용하기 위한 헤더
- `#include <assert.h>`: 개발 중 조건 검증을 위한 `assert()` 매크로 사용을 위한 헤더
- `#include <unistd.h>`: 시스템 콜을 위한 헤더로, 실제 힙 확장에 사용되는 `sbrk()`와 관련된 기능을 제공
- `#include <string.h>`: 메모리 블록 복사나 초기화 등의 작업을 위한 헤더로, `memcpy()`, `memset()` 등의 함수를 제공
- `#include "mm.h"`: 할당기 구현을 위한 인터페이스가 정의된 사용자 정의 헤더 파일로, `mm_init()`, `mm_malloc()`, `mm_free()` 등의 함수 프로토타입과 제출자 정보를 담는 `team_t` 구조체가 선언되어 있다.
- `#include "memlib.h"`: 가상 메모리 모델을 제공하는 프로젝트용 라이브러리로, `mem_sbrk()`, `mem_heap_lo()`, `mem_heap_hi()` 등 가상 힙 제어 함수들이 선언되어 있다.

3.2 매크로, 구조체, 전역변수

① 정렬 관련 매크로

- `#define ALIGNMENT 8`: 정렬 기준을 8 바이트로 설정
- `#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~0x7)`: 주어진 `size` 를 8 의 배수로 올림 처리

② 블록 크기 및 메타데이터 크기 관련 매크로

- `#define WSIZE 4`: 블록의 헤더 또는 푸터 크기 (4 바이트, 워드 단위)
- `#define OVERHEAD 8`: 헤더(4B) + 푸터(4B)의 합, 즉 하나의 블록에서 메타데이터가 차지하는 공간
- `#define DSIZE 8`: 더블 워드 크기 (8 바이트), 최소 블록 단위 연산 기준
- `#define PTR_SIZE sizeof(void *)`: 포인터 크기, 64 비트 시스템에서 보통 8 바이트
- `#define MAX_FREELISTS 20`: segregated free list 의 총 개수를 20 개로 설정
- `#define MIN_BLOCK_SIZE 32`: free block 이 가져야 할 최소 크기 (헤더, 푸터, prev, next, padding 포함)
- `#define INITIAL_HEAP_SIZE (1<<11)`: 초기 힙 크기를 $2^{11} = 2048$ 바이트로 설정
-

③ 블록 HEADER/FOOTER 조작 매크로

- `#define PACK(size, alloc) ((size) | (alloc))`: size 와 할당 여부 비트를 결합하여 하나의 워드로 만듦
- `#define GET(p) (*(unsigned int *)(p))`: 주어진 주소 p 에서 워드 단위 값을 읽음
- `#define PUT(p, val) (*(unsigned int *)(p) = (val))`: 주어진 주소 p 에 워드 단위 값 val 을 저장
- `#define GET_SIZE(p) (GET(p) & ~0x7)`: 헤더 또는 푸터에서 블록의 크기만 추출 (하위 3 비트 제거)
- `#define GET_ALLOC(p) (GET(p) & 0x1)`: 헤더 또는 푸터에서 할당 여부 비트(LSB)만 추출

④ 블록 포인터 연산 관련 매크로

- `#define HEADER_PTR(bp) ((char *)(bp) - WSIZE)`: payload 포인터로부터 헤더 주소 계산
- `#define FOOTER_PTR(bp) ((char *)(bp) + GET_SIZE(HEADER_PTR(bp)) - DSIZE)`: payload 포인터로부터 푸터 주소 계산
- `#define NEXT_BLOCK_PTR(bp) ((char *)(bp) + GET_SIZE(((char *)(bp) - WSIZE)))`: 다음 블록의 payload 주소 계산
- `#define PREV_BLOCK_PTR(bp) ((char *)(bp) - GET_SIZE(((char *)(bp) - DSIZE)))`: 이전 블록의 payload 주소 계산

⑤ Free List 연결 포인터 접근 매크로

- `#define PREV_NODE(bp) (*(char **)(bp))`: segregated list 에 있는 node 의 prev 포인터 접근
- `#define NEXT_NODE(bp) (*(char **)((bp) + PTR_SIZE))`: segregated list 에 있는 node 의 next 포인터 접근

3.3. 전역 변수

- `static void **segregatedFreeLists`: 크기 범위에 따라 분리된 20 개의 segregated free list 를 가리키는 포인터 배열. 각 list 는 크기별로 정렬된 이중 연결 리스트의 head 역할을 함
- `team_t team`: 제출자의 학번, 이름, 이메일 정보를 담는 구조체.

3.4 Subroutine

① `static void *expand_heap(size_t size)`

힙을 size 만큼 확장하고, 확장된 공간을 free block 으로 초기화한다.

- 동작 방식:
 1. 요청 크기를 8 바이트 정렬
 2. `mem_sbrk()`로 실제 힙 공간을 확장
 3. 헤더/푸터 설정 후 epilogue 재설정
 4. free list 에 삽입 후 `coalescing_blocks()`로 병합
- Return value
 1. 새로 생성된 free block 의 payload 포인터를 반환

2. 힙 확장에 실패하면 NULL 반환

② static void *allocate_block(void *bp, size_t alignedSize)

주어진 free 블록을 size 에 맞게 할당하거나 분할하여 할당한다.

- 동작 방식:
 1. 블록 크기와 요청 크기를 비교
 2. 크기 차이에 따라 다음 중 하나 실행:
 - 전부 할당
 - 앞쪽은 free list 에 추가하고 뒤쪽 메모리를 할당
 - 앞쪽 메모리를 할당하고, 나머지는 free list 에 추가
 3. header/footer 설정 후 반환
- Return value
크기에 따라 할당 완료된 블록의 payload 포인터를 반환

③ static void *coalescing_blocks(void *bp)

주어진 free block 을 기준으로 앞뒤 블록이 free 이면 병합(coalescing)한다.

- 동작 방식:
 1. 이전, 다음 블록의 할당 상태 확인
 2. 각각의 경우(2-way, 3-way, 병합 없음)에 따라 병합 수행
 3. 병합 전 블록들은 free list 에서 제거
 4. 병합된 블록을 free list 에 다시 삽입
- Return value
 1. 병합 후의 블록 payload 포인터를 반환
 2. 병합이 일어나지 않으면 입력된 bp 그대로 반환

④ static void insert_into_free_list(void *bp)

free 상태의 블록을 적절한 segregated list 에 삽입한다.

- 동작 방식:
 1. 블록 크기에 따라 리스트 인덱스를 결정
 2. 해당 리스트에서 오름차순 정렬 유지하며 위치 탐색
 3. 삽입될 위치에 따라 prev/next 포인터 설정하여 이중 연결 리스트에 삽입
- Return value
이 함수는 내부 상태 변경만 수행하고, 값을 반환하지 않음

⑤ static void remove_from_free_list(void *bp)

free list 에서 특정 블록을 제거한 후, 사용자에게 해당 블록을 반환해 할당할 메모리를 전달해준다.

- 동작 방식:
 1. 블록 크기로 리스트 인덱스를 결정
 2. 해당 리스트에서 prev/next 포인터를 조정하여 연결 해제
 3. 리스트에 bp 만 있는 경우, head 를 NULL 로 초기화

- Return value
이 함수는 내부 상태 변경만 수행하고, 값을 반환하지 않음

⑥ void *find_fit_block(size_t size)

요청 크기 이상의 블록을 free list 에서 first-fit 방식으로 탐색한다.

- 동작 방식:
 1. size 가 속하는 리스트부터 순차적으로 탐색
 2. 오름차순 정렬된 리스트에서 첫 번째로 맞는 블록 반환
 3. 못 찾으면 NULL 반환
- Return value
 1. 요청 크기를 만족하는 free block 의 payload 포인터 반환
 2. 적절한 블록이 없으면 NULL 반환

각 함수와 매크로는 포인터 연산 및 메모리 정합성 유지를 위해 상호 유기적으로 연결되며, 할당기의 성능(특히 Utilization 과 Throughput)에 큰 영향을 준다.

4. 최적화 전략

메모리 단편화를 줄이고 탐색 속도를 향상시키기 위해 다음과 같은 전략을 적용하였다.

5.1 Segregated Free List

- 블록 크기에 따라 free list 를 나누어 **탐색 범위 축소**
- 작은 블록은 작은 리스트에서, 큰 블록은 큰 리스트에서만 검색 → 성능 향상
- 총 20 개의 리스트를 사용하고 있으며, 각 리스트는 다음 범위로 분류됨:

List Index	Block Size 범위
Index 0	1 ~ 32 bytes
Index 1	33 ~ 64 bytes
Index 2	65 ~ 128 bytes
...	...
Index 19	2 ¹⁹ bytes (최대 크기)

5.2 First-Fit with Size-Ordered Insert

- 각 리스트 내부는 블록 크기 기준 오름차순으로 정렬
- 오름차순으로 정렬하였으므로 탐색 시 가장 앞에서부터 검색하여 첫 번째 적합 블록을 사용하는 것이 가장 optimal 한 선택이 된다.

5.3 Block Splitting

- 큰 블록을 할당할 때는 분할(splitting)을 고려하여 내부 단편화 최소화
- 남는 공간이 240B 이하일 경우, 앞쪽을 free 로, 뒤쪽을 할당 영역으로 사용
- 240B 초과 남을 경우, 뒤쪽을 free 로 유지하여 재사용 가능하게 함

5.4 Coalescing (Block Merging)

- free() 또는 expand_heap() 이후 인접한 free block 이 존재할 경우, 최대 3-way 병합 수행
- 병합된 블록은 다시 free list 에 삽입되어 재활용 가능

5.5 In-place Reallocation

- realloc() 구현 시, 만약 요청한 사이즈가 기존에 할당되었던 사이즈보다 작다면 원래 사용하던 block 그대로 return
- 또, 만약 뒤 블록이 free 이고 충분한 크기일 경우 새로운 메모리 할당 없이 기존 블록과 free block 을 병합한 block 을 return
- 새 블록을 할당하고 데이터를 복사하는 비용을 줄이고, throughput 향상에 기여

5.6 Block Alignment & Minimum Block Size

- 8 바이트 정렬을 유지하여 포인터 연산의 안전성 확보
- 최소 블록 크기(32B)를 설정하여 prev, next 포인터 및 header/footer 보장

이러한 최적화 전략들은 utilization 과 throughput 간의 균형을 고려하여 설계되었으며, 평가 도구인 mdriver 의 성능 지표(Performance Index)를 최대화하는 데 중점을 두었다.

6. 성능 측정 결과 요약

할당기의 성능은 mdriver 프로그램을 사용하여 평가되었으며, correctness, space utilization, throughput 을 기준으로 측정되었다. 평가 시 -v 또는 -V 옵션을 활용하여 trace 별 결과 및 전체 지표를 분석하였다.

6.1 실행 환경

- OS: cspro 서버 환경
- Compiler: gcc
- 컴파일 옵션 (CFLAGS)
 - -Wall: 모든 경고 메시지를 출력
 - -O2: 최적화 수준 2 로 설정 (성능 개선)
 - -m32: 32 비트 모드로 컴파일 (x86 호환 환경에서 실행 가능하도록 설정)
- 컴파일 및 실행 command: 주어진 Makefile 로 compile
make
./mdriver -v

6.2 실행 결과

```
cse20200595@cspro:~/SP/Project4/prj4-malloc$ ./mdriver -v
[20200959]::NAME: Seokhee Lee, Email Address: lshlkh1234@naver.com
Using default tracefiles in ./tracefiles/
Measuring performance with gettimeofday().

Results for mm malloc:
trace  valid  util    ops      secs  Kops
0       yes   99%    5694  0.000455 12523
1       yes   99%    5848  0.000687  8509
2       yes   99%    6648  0.000797  8339
3       yes   99%    5380  0.000676  7955
4       yes   88%   14400  0.000820 17567
5       yes   95%    4800  0.001497  3206
6       yes   94%    4800  0.001567  3062
7       yes   84%   12000  0.002610  4597
8       yes   78%   24000  0.003354  7155
9       yes   28%   14401  0.000814 17692
10      yes   67%   14401  0.000601 23970
Total                85%  112372  0.013879  8096

Perf index = 51 (util) + 40 (thru) = 91/100
cse20200595@cspro:~/SP/Project4/prj4-malloc$
```

- trace 0~3 은 매우 높은 utilization(99%)과 안정적인 처리량을 기록
- trace 4~6 은 85~95%로 약간 낮은 수준의 메모리 활용도
- trace 7, 8, 10 에서는 utilization 이 다소 감소
- trace 9 에서는 낮은 performance 를 보여주었다.